

Billy Ocean keyboard bass patches and Logic Pro synth plugins

Shared by Andy

 This is a copy of a chat between Claude and Andy. Content may include unverified or unsafe content that do not represent the views of Anthropic. Shared snapshot may contain attachments and data not displayed here.

[Report](#)

Can you find out what sort of keyboard bass patches were used on the Billy Ocean Caribbean Queen album, they all seem the same. And are there any free or inexpensive plug ins for Logic Pro to experiment with creating them? They seem like fairly standard synthesizer sounds.

Searched the web

Here's what the credits actually tell us, plus some informed inference about the gear.

What the album credits say

The album (officially *Suddenly*, 1984 — "Caribbean Queen" is the lead single) doesn't list specific synth models, but the credits are telling: producer Keith Diamond employed synthesizers for bass lines and drum machines such as the LinnDrum and Simmons SDS-V for the electronic percussion sound, with Barry J. Eastmond on keyboards/synths and Fairlight CMI programming by J.J. Jeczalik and Pete Harris. Keith Diamond is repeatedly credited as "Synth Bass" across the various pressings, separate from Timmy Allen's live bass guitar credit. [Grokikipedia](#)

So the "samey" bass patch you're hearing across the album is almost certainly **one core synth bass sound that Keith Diamond reused as his signature patch**, likely layered/doubled with Timmy Allen's real bass guitar on some tracks for weight.

What synth was it, most likely?

Nobody published the exact patch, but given the era (1983–84, New York session scene, this specific sonic signature — tight, slightly clicky, sustained mid-low synth bass with a fast attack and a bit of pitch glide/portamento), the standard suspects are:

- **Sequential Circuits Prophet-5** — the workhorse analog bass of NYC sessions at

the time

- **Oberheim OB-8 / OB-Xa** — very common for that punchy R&B/pop synth bass
- **Yamaha DX7** — released in 1983, instantly ubiquitous; its factory "Bass 1/Bass 2" FM patches have that clicky, slightly metallic low end you hear all over 1984 pop
- **Fairlight CMI** — since it's credited for programming, some of the bass hits may even be Fairlight-sampled sounds rather than a synth oscillator at all

Given the timing (DX7 came out in '83 and was *everywhere* by mid-'84) and the description of a clean, repetitive, slightly percussive bass tone, my best guess is a **DX7 FM bass patch, possibly doubled with an analog synth (Prophet-5 or Oberheim) for body**, with the Fairlight adding transient/percussive character.

Logic Pro plugins to experiment with this sound

You don't need anything exotic — this era of bass patch is very achievable.
Free/cheap options in Logic:

Already in Logic (free, no plugin needed):

- **ES2 or EFM1** — EFM1 is Logic's built-in FM synth, perfect for DX7-style bass (try a sine-ish operator with a fast-decay second operator for the "click")
- **Retro Synth** (FM mode) — simplified FM, fast way to dial in DX7-ish bass
- **ES1/ES2** in analog mode for Prophet/Oberheim-style fat bass — sawtooth or pulse wave, fast attack, short-medium decay, low filter cutoff with some resonance

Free third-party:

- **Dexed** (free, open source) — a faithful DX7 emulator with patch libraries online, including actual "BASS1"/"BASS2" factory patches you can load directly. This is probably your fastest path to the exact sound.
- **Surge XT** (free) — capable hybrid synth, good for both FM and subtractive bass experiments
- **TAL-NoiseMaker** (free) — simple analog-style synth, decent for OB-8/Prophet-ish bass tones

Cheap/affordable:

Cheap/available:

- **u-he TyrellN6** (free) or **u-he Diva** (paid, ~\$179 but frequently discounted, very highly regarded for accurate vintage analog modeling — Oberheim/Prophet/Jupiter modes built in)
- **Arturia's Mini V / OB-Xa V** if you ever want the real deal modeled — pricier but exact emulations exist in their V Collection

If you want to nail the *specific* "Caribbean Queen" tone, I'd start with Dexed loaded with a classic DX7 "BASS 1" patch, then A/B against ES2 set to a Prophet-style analog patch, and see which one matches what you're hearing — since it may even be a layer of both.

The patch doesn't sound like DX7, I'd guess Oberheim or Prophet from that list. It's kind of fat with a filter sweep

That fits much better — a DX7 bass is usually clean and percussive without movement, while a filter sweep is a classic analog subtractive trait (envelope modulating the cutoff). Oberheim OB-8/OB-Xa and Prophet-5 were both go-to "fat sweeping bass" sources in 1983-84 NYC sessions, so that's a solid guess.

Here's how to dial that in:

Core patch recipe (works in ES2, ES1, Diva, TAL, Surge, etc.):

- **Oscillators:** 2 sawtooth waves, one detuned slightly (5-10 cents) for width/fatness. Optionally add a sub-oscillator an octave down for weight.
- **Filter:** Low-pass, 24dB/oct (4-pole) — this is key to the Oberheim/Prophet character vs. the thinner 12dB filters on some other synths.
- **Filter envelope:** Fast attack, then a sweep down — start cutoff high/open, decay down to a lower sustain level over maybe 200-500ms. That's your "sweep."
- **Filter envelope amount:** Push it fairly hard so the sweep is obvious, then back off to taste.
- **Resonance:** Small-to-moderate amount — enough to give the filter some bite as

it sweeps, not so much it self-oscillates.

- **Amp envelope:** Fast attack, medium decay, low-ish sustain, short release — typical bass shape.
- **Glide/portamento:** A little bit (10-30ms) can add that "Diamond" slidey character between notes if you hear pitch glide too.

Best Logic-native option: ES2 is genuinely good for this — it has dual filters and enough modulation routing to get a convincing fat analog sweep. Set both oscillators to saw, route the filter envelope generously, use the 24dB low-pass mode.

Best free third-party for accuracy: TAL-NoiseMaker or u-he TyrellN6 — both are modeled more on classic analog subtractive architecture than FM, so they'll get you there faster than Dexed will for this particular sound.

If you want the closest possible match: u-he Diva (paid, ~\$179, often on sale) has dedicated Oberheim and Prophet-5 filter models you can select directly — it's about as close as software gets to the real hardware's nonlinear filter behavior, which is a lot of what gives that "fat" quality.

Want me to sketch out actual parameter values for ES2 specifically, since you've got that free in Logic already?

Sure

Here's a starting point dialed into ES2's actual interface:

Oscillators (top section)

- Osc 1: Saw wave
- Osc 2: Saw wave, detune ~+7 cents (use the Mod wheel/Saturator section's fine-tune or the Beat Detune knob if available)
- Osc 3: Off, or set as a sub-sine an octave down, mixed in low (~20-30%) for weight
- Mix balance: roughly equal Osc 1/2, sub-osc subtle underneath

Filter section

- Filter type: Fil 1, select **24dB low-pass (4-pole)** — this is the slope that gets you the fatter, more "vintage" character vs. the thinner 12dB option
- Cutoff: start around 30-40% (you want headroom for the envelope sweep to open it up)
- Resonance: 15-25% — enough for some bite, not self-oscillating
- Drive/Saturation if available: a touch, for analog grit

Filter Envelope (Env 2, routed to cutoff)

- Attack: 0 (instant)
- Decay: 200-400ms
- Sustain: fairly low, 10-20%
- Release: short, 50-100ms
- Env 2 → Cutoff amount: push this fairly high (50-70%) so the sweep is obvious — you can always pull it back once you hear it

This combo is what creates the audible "sweep": cutoff snaps open on the attack, then decays down to a lower sustain level as the note holds.

Amp Envelope (Env 1)

- Attack: 0
- Decay: 150-300ms
- Sustain: 40-60%
- Release: 80-150ms

Glide

- Turn on Glide/Portamento, set to ~15-25ms, mode "Legato" if available (so it only glides between overlapping/tied notes, not every note) — gives you that slidey character without making every note mushy

Global

- Pitch: drop an octave or two depending on register
- Try the low-pass at 24dB first; if it still feels thin, increase resonance slightly or push the filter envelope amount further before adding more oscillator layers

Play a simple repeated bass line and tweak the **Env2→Cutoff amount** and **Decay time** first — that's 80% of what makes it feel like "the" sound. Once that's locked in, the detune and resonance are just polish.

It doesn't look I select ES2 from the instrument type in Logic Pro?

That's usually just a navigation thing, not a missing plugin — ES2 ships with every full version of Logic Pro. A few things to check:

1. **Where to look:** Click the instrument slot on a software instrument track → it should show a menu with categories like *Logic > Synthesizer*. ES2 is normally filed under **AU Instruments > Logic > Synthesizer > ES2**, not under a generic "Logic" folder at the top level.
2. **Check you're on a Software Instrument track**, not Audio or Drummer — ES2 only shows up as an option there.
3. **If you genuinely don't see it at all:** Logic Pro has been slowly retiring some legacy synths in favor of newer ones in recent versions — it's possible your install has ES2 hidden or it's been nudged toward **ES1, ESM, or Alchemy** as default options depending on version/template. Apple did fold some EXS/ES-series synths' functionality into newer plugins over recent updates.
4. **Quick fix if missing:** Go to **Logic Pro > Sound Library > Download All Available Sounds** (or check **Logic Pro > Settings > Plugin Manager** and rescan/reset) — sometimes synths show as available but aren't installed/indexed until the sound library content downloads.

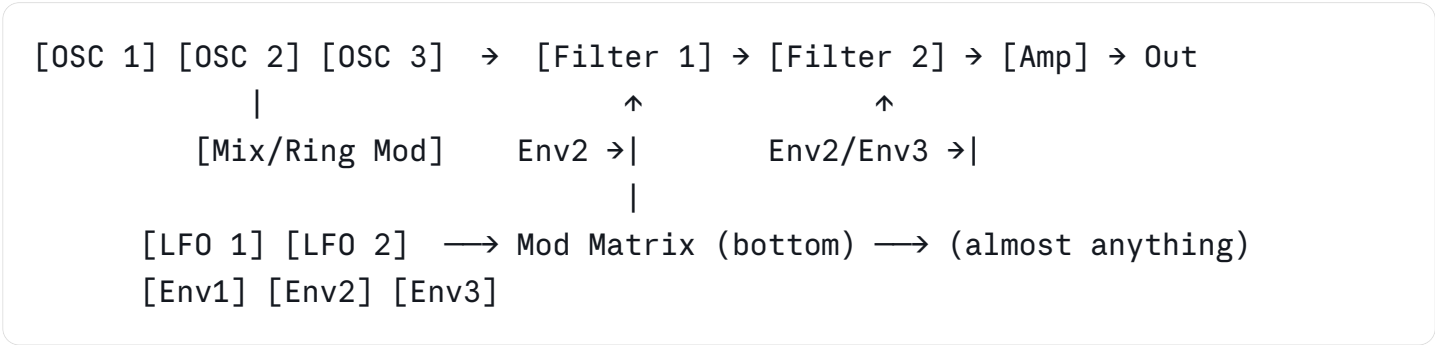
Can you tell me what you do see in that instrument list when you click the track's instrument slot? That'll tell me whether it's a filing/category issue or an actual missing-plugin issue, and I can point you to the exact spot — or to the best alternative

if ES2 truly isn't there in your version.

Found it. ES2 is a maze, and it seems there are a lot of redundant controls. I'm very familiar with synth architectures as an engineer, but I need a bit of a map. I did get pretty close pretty quick

Fair complaint — ES2 crams a lot in and reuses panels for different purposes depending on mode. Here's the actual signal flow and where things live:

Signal flow (top to bottom, left to right):



Why it feels redundant — three independent modulation paths overlap:

- 1. **Hardwired routings** (the visible knobs near filter/amp — Env2→Cutoff, Env1→Amp) — fixed, always-on
- 2. **The Mod Matrix** (bottom section, easy to miss) — fully flexible source→destination routing, *additional* to the hardwired stuff
- 3. **The Vector Envelope** (top-left triangle pad) — a 4-corner morphing system blending Osc1/2/3 + Filter, mostly for evolving pad sounds — **ignore this entirely for bass**, it's the main source of confusion and irrelevant to what you're doing

For your purposes, the panels that actually matter:

Panel	What it does	For bass patch
Oscillator section	Waveform, octave, semi, detune for Osc 1/2/3	Set 2 saws + sub, as discussed

(top)

Filter section (Fil 1 / Fil 2)	Cutoff, resonance, slope (12/24dB), Fil1→Fil2 routing (series/parallel)	Use Fil 1 only, 24dB mode, ignore Fil 2
Filter routing switch	Series vs. parallel vs. Fil2-only	Leave on default series, Fil2 bypassed/not driven
Envelopes (Env1/2/3)	Env1 = hardwired to Amp. Env2 = hardwired to Filter (this is your sweep). Env3 = free, assign via Mod Matrix if needed	You only need Env1 + Env2
Mod Matrix (bottom)	Free-form routing table	Ignore for now — you don't need it for this patch
LFOs	Vibrato/tremolo/filter wobble	Ignore for bass unless you want movement later
Vector envelope / joystick	Morphs between oscillator/filter states over time	Ignore completely — this is the rabbit hole

So practically: the only panels you need are **Oscillators → Filter 1 → Env 2 (filter sweep) → Env 1 (amp shape)**. Everything else — Fil 2, the vector pad, the mod matrix, LFOs — is for more complex evolving sounds and can be left at default/off. That's most of the "maze" feeling resolved right there: ES2 is really three synths bolted together (subtractive, vector, FM-ish via ring mod), and you only need the basic subtractive third of it.

There is a round filter section in the middle that seems redundant with the ADSRs. Is the "cut" knob there just an offset and the ADSR adds to

it? Multiplies it?

Additive, not multiplicative — that's true for essentially all subtractive synth filters, including ES2's.

How it actually works: The Cutoff knob sets a base frequency value. Every modulation source (Env2 amount, velocity, key tracking, LFO, mod wheel via the matrix) adds or subtracts an offset from that base point, all summed before hitting the actual filter. Think of it as the knob position being "zero" on a modulation bus, and each source nudging it up or down from there — not scaling it as a percentage.

That's why the **Env2 amount knob** in that circular section matters so much: it's not "how much of the envelope to let through," it's literally "how many semitones/units of cutoff offset to add at the envelope's peak." Turn the base Cutoff down low and crank Env2 amount up high, and you get a huge sweep from closed to open. Turn Cutoff up high with low Env2 amount, and you barely hear the envelope at all even though it's technically still modulating.

Practical implication for your patch: this is exactly why the "sweep" character is a *relationship* between two knobs, not one setting — Cutoff (base) and Env2 amount (offset depth) trade off against each other to get the same perceived sweep range. If your sweep currently sounds too narrow, you have two options that do different things tonally:

- Lower the base Cutoff and keep Env2 amount the same → sweep starts from a darker resting point, same peak
- Raise Env2 amount and keep base Cutoff the same → sweep goes higher/brighter at the peak, same starting point

Velocity and key-tracking in that same circular cluster work identically — additive offsets, not multipliers — so a hard-hit note adds its own offset on top of whatever the envelope is doing at that instant, which is why velocity-sensitive filter patches can feel inconsistent if you're not used to thinking in "all offsets sum together" terms rather than "velocity scales the envelope."

The three oscillator section on the left is a little confusing as well. There is some kind of LFO effect by default that is causing the low end to gradually be weaker and stronger, which is not good. I suspect that has something to do with this

That's almost certainly the **Vector Envelope** doing exactly what I told you to ignore — and it's worth circling back to because it's likely on by default in whatever patch you started from.

What's happening: the vector envelope can have a "loop" function that continuously animates the X/Y vector point across the triangle pad over time, automatically crossfading the balance between Osc 1/2/3 (and sometimes filter balance) on a cycle. If that loop is active, it'll create exactly the slow "breathing" effect you're describing — the low end (your sub-osc or one of the saws) gradually fading in and out as the vector point sweeps back and forth.

Where to check/kill it:

1. Look at the vector envelope section (the triangle/joystick pad, usually labeled with the three oscillator destinations at its corners)
2. There's a small **Loop** button/toggle near that section — if it's lit/engaged, that's your culprit. Turn it off.
3. Also check the vector envelope's own rate/time controls — even without "loop," if it has a multi-stage envelope set with a slow attack/decay between corners, it'll still cause a one-time (or repeating) drift in oscillator balance.
4. If you want to be fully sure it's not interfering at all, the cleanest move is to **set the vector point to a fixed position** (drag it to a corner or center and leave it static) rather than just muting the loop — that guarantees no automatic morphing regardless of envelope/loop state.

This is the exact rabbit hole I flagged earlier — ES2's default factory patches often have the vector animation running because it's a flashy, demo-friendly feature, but for a straightforward bass patch it's pure noise. Killing the loop and pinning the vector point should make your low end rock solid.

I don't think so, because the matrix switch is on

Good catch — that's actually more likely to be the real cause, and it's an easy thing to overlook.

What "the matrix switch is on" means: ES2's Mod Matrix is a grid of source→destination routing slots, each with its own on/off enable and an amount knob. If one of those rows is active and routed to something like Osc Mix, Volume, or even Cutoff, an LFO or envelope can be pulsing that destination independently of anything in the vector or filter sections — which would explain the periodic strengthening/weakening you're hearing in the low end specifically.

To find and kill it:

1. Open the **Mod Matrix** panel (bottom of ES2, the table with Source/Via/Destination/Amount columns)
2. Look through each row for one that's **enabled** (usually a small on/off toggle or non-zero amount) with a **Source = LFO1 or LFO2**
3. Check what the **Destination** column says — if it's set to **Mix** (oscillator balance), **Volume**, or **Sub Osc level**, that's your slow pulsing low end
4. Either toggle that row off, or zero out its Amount knob

Quick diagnostic if you don't want to hunt row by row: just mute LFO1 and LFO2 outright (there's usually a rate/reset area near each LFO — turning the rate to 0 or finding a bypass works too), then play a note. If the pulsing stops, you've confirmed it's an LFO routed through the matrix rather than anything in the vector section, and you can re-enable the LFO once you've identified and disabled the specific offending row.

One more thing worth checking once it's fixed: make sure that same matrix row isn't something you actually want later (LFO-to-filter for movement, etc.) — better to zero the amount than fully disable the row, so you can dial it back in deliberately if you want subtle movement instead of the current overly-strong default.

Why are there three oscillators?

Three oscillators in ES2 isn't really about "more is better" stacking — it's there specifically to feed the **vector mixing** system you've been fighting with.

The actual design intent:

ES2's architecture is built around the vector envelope/joystick having multiple corners to morph between. With 3 oscillators (plus a separate filter balance axis), you get a triangular or 4-point space where the vector position determines the mix ratio between them in real time. That's the synth's signature feature — it's meant for evolving timbral movement (e.g., morphing from a sine sub, to a saw, to a noisy/FM-ish third oscillator, and back), which is great for pads and sound design, not so useful for a static bass patch.

Secondary reasons for 3, beyond vector morphing:

- **Osc 3 has extra modes** that Osc 1/2 don't — it can act as a **noise source**, or be used for **ring modulation** against Osc 1, or run in **sync mode** — so it's partly there as a "utility" oscillator for texture/aggression, not just a third voice
- Having 3 independent oscillators with separate octave/semi/detune also just gives more layering options for fatness (your earlier sub-osc idea), even without invoking the vector system at all

For your bass patch specifically: you're using 3 oscillators only as static layers (saw, detuned saw, sub), with the vector point pinned so it's not morphing between them — which is a completely valid and common way to (under)use ES2. You're essentially using 1/3 of what the oscillator section is capable of, and ignoring the modulation/morphing half entirely. That's fine — it's the same approach a lot of people take when using ES2 as "just a subtractive synth" rather than engaging its more unique vector-synthesis side.

Is Claude responding

Yes, I'm here! Did my last message not come through fully, or did you have a follow-up question about the oscillators?

Claude was unresponsive for a while

Sorry about that — occasional slowdowns do happen on Anthropic's end, especially at busy times. I'm responding normally now though.

Did you want to carry on with the ES2 patch, or did you have another question?

I've moved onto another question. I've been considering emulating keyboard Bass based on the signal from a bass guitar. There was a period of time after an attack where we don't know the frequency, that's the tricky part. I was thinking of using non-linear wave shaping during that initial attack or some generation based on the early peaks. But I know from experiencing that nonlinear processing introduces overtones that can alias back into the passband. Other than over sampling like Line 6 does are there other techniques to consider?

Great problem — you're essentially describing pitch detection latency vs. transient fidelity, which is a classic tradeoff. A few approaches worth considering beyond oversampling:

On the aliasing problem specifically:

- **Bandlimited waveshaping** — instead of applying a static waveshaper to the raw

signal, apply it only to a bandlimited version (steep low-pass first, shape, then reconstruct). You lose some of the transient "energy" above the fundamental but the aliases stay out of the passband. The tradeoff is it softens the attack characteristic you're actually trying to use.

- **Antiderivative antialiasing (ADAA)** — a more elegant analytical approach where you apply the antiderivative of your waveshaping function to cancel alias components mathematically rather than filtering them out. It's computationally cheap relative to oversampling and works surprisingly well for moderate nonlinearities. Matt Tytel (Vital synth) has written about this.
- **Minimum-phase vs linear-phase filtering** matters here too — if you're oversampling at all, a minimum-phase decimation filter adds less smearing to the transient than a linear-phase one, which preserves more of the attack character you're trying to exploit.

On the pitch-detection-during-attack problem:

- **Autocorrelation on early partial peaks** rather than waiting for zero-crossing stability — you may get a reliable enough estimate from just 1-2 cycles of the dominant partial even in the noisy attack region
- **Confidence gating** — rather than outputting "wrong" pitch data during the attack, hold the previous known pitch (from the tail of the previous note) and blend into the new pitch only once confidence exceeds a threshold. Works well for bass because the attack is often harmonically rich but rhythmically predictable
- **Spectral flux as a trigger** — use the onset/transient detector not for pitch but purely as a "new note" flag, then let pitch tracking catch up 10-20ms later. During that window, play a synthesized transient burst (noise burst shaped like a bass attack) that's pitch-neutral, blending into the pitched oscillator as the tracker locks in. Essentially decoupling the transient from the pitched content.
- **MIDI or parallel pickup as a cheat** — if you have any control over the hardware side, a hexaphonic pickup or even a piezo saddle gives you per-string onset timing that sidesteps the monophonic pitch-detection-during-attack problem entirely

The deeper architectural question is whether you're doing this in the time domain

or frequency domain. A phase vocoder / STFT approach gives you much better frequency resolution during the attack (via shorter hop sizes) at the cost of some time smearing, whereas time-domain approaches get the transient right but struggle with the ambiguous frequency content you described. Some modern pitch trackers (like pYIN) use a probabilistic model that hedges between multiple pitch candidates during uncertain regions rather than committing early, which might give you better behavior to "drive" your synthesis from.

What's your current approach for the pitch detection itself — autocorrelation, zero crossing, FFT-based, or something else? That would shape which of these is most practical to bolt on.

Bass guitar has low frequencies. I've got a pitch detector working reliably based on yin, but it takes three cycles or so to Lock during that initial period of time I need to generate some other sound that's as close as possible to what will be picked up once the frequency locks up that's the trick. And unfortunately, from the experimentation, I've been doing with some of the synthesizers I would like to emulate, these oscillator signals that are being filtered are very rich in harmonics. Such as sawtooth waves.

Right, so the problem is specifically that a sawtooth through a resonant low-pass (which is exactly what you've been building in ES2) has a lot of harmonic content that the alias products from nonlinear shaping will collide with — it's not a clean sine you're trying to approximate, it's a spectrally dense target.

A few thoughts on the "fill the gap" problem specifically:

The core insight: you don't need to be *correct* during those 3 cycles, you need to be *plausible and continuous* so the transition into the locked pitch isn't jarring. That's a softer constraint than it sounds.

Approaches worth considering:

Pitch carry-forward with envelope freshening

Hold the last known pitch from the previous note, immediately retrigger your synth envelope on onset detection (even before pitch locks), then crossfade into the new pitch when YIN commits. If the bass player is playing in a normal range and not jumping wildly, the "wrong" pitch during the gap is often close enough that the envelope shape masks it — the listener hears the attack transient and envelope sweep, not the specific frequency.

Spectral envelope matching during the gap

During the attack, rather than trying to synthesize the correct pitch, synthesize a spectrally *shaped* noise burst or sawtooth at a "safe" frequency (the previous note, or a median register guess) but shaped to match the spectral envelope of the incoming signal via a fast adaptive filter. The idea being that the harmonic *density and tilt* matters more perceptually during a transient than the exact fundamental.

Sub-band splitting

Split the incoming signal into low (fundamental region) and high (harmonics/attack noise) bands. During the lock period, pass the high band through relatively unprocessed or lightly shaped — it contains the attack character and is harder to pitch-perceive anyway. Only apply your synthesis/waveshaping to the low band once YIN locks. This sidesteps the aliasing problem during the attack because you're not nonlinearly processing the high band.

Onset-triggered noise + fast filter sweep

A pitched noise burst (bandpass filtered around the expected fundamental region, shaped with a very fast decay) during the attack gap sounds surprisingly like a bass attack transient to the ear — the initial "thud" of a bass note is largely noise-like anyway. Blend that into your oscillator as YIN locks. This is essentially what a lot of physical modeling systems do for the initial excitation.

On the aliasing problem with sawtooth targets:

The fundamental issue is that if you're generating a sawtooth to emulate the synth, you need it bandlimited from the start — a BLEP (bandlimited step) or BLAMP sawtooth oscillator rather than a naive one avoids generating aliases above Nyquist in the first place, which is architecturally cleaner than trying to filter them out post-waveshaping. BLEP is well documented and not expensive to implement. That gets you a clean sawtooth to work with before any filtering happens.

The transition itself is probably where the most polish comes from — a very short

crossfade (even 10-15ms) between your gap-filler and the locked oscillator, timed to coincide with a zero crossing in both signals if possible, will hide a lot of sins. The ear is remarkably forgiving if the transition is fast and buried in the decay of the attack transient.

What's your implementation environment — DSP in C/C++, something like JUCE, or a higher level context?

I've created an embedded ecosystem using Claude code. The DSP code is C plus plus and Faust. What you're missing in this problem is that three cycles of a bass guitar wave is a whole lot more than 15 ms that's more like one cycle and that's not even the lowest note. Admittedly, I probably wouldn't be using the lowest notes as triggered still it's a pretty big gap. Otherwise I'd be a little more confident than noise burst type of idea would work. I've been thinking about trying to reconstruct the first few cycles, using the actual way form from the Bass Guitar.

You're absolutely right, I was sloppy with the timing — at low E (41Hz) three cycles is ~73ms, and even at A2 (110Hz) it's ~27ms. That's a very audible gap, not a cosmetic one. The noise burst idea falls apart at that timescale.

Your waveform reconstruction idea is the right direction. Here's why it's promising and what the challenges are:

The core idea — using the actual bass waveform:

During the pre-lock period you have real samples coming in. Even without knowing the period yet, you have the actual attack transient of *that specific note* from *that specific instrument*. The insight is that the bass guitar's own waveform during the attack is already "correct" — it just needs to be continued or morphed into the synthesized target rather than replaced.

Practical approaches to reconstruction:

Epoch-based extension

Once YIN locks (even partially, with low confidence), you have a period estimate.

You can then look back into your ring buffer, find the most recent complete cycle, and loop it forward while simultaneously blending into your synthesized oscillator. The looped real waveform is spectrally correct by definition — it came from the instrument.

Predictive period estimation from partials

Even during the attack, the higher harmonics stabilize faster than the fundamental — a 500Hz partial of a 100Hz note completes a cycle in 2ms, well within your first few ms of signal. You could run a parallel high-frequency autocorrelation or comb filter bank on the early signal to get a period estimate faster than YIN can from the fundamental alone, then use that to seed your reconstruction earlier.

Gradient-based waveform morphing

Rather than a crossfade, morph the real bass waveform *gradually toward* the sawtooth+filter target over several cycles once pitch locks. Match them at a zero crossing, then over 3-5 cycles progressively blend the spectral content from "real bass" toward "synthesized target." The ear accepts gradual spectral change much better than a sudden switch.

The harder problem you're implicitly facing:

The real bass attack waveform and your synthesized sawtooth-through-filter are *spectrally quite different* — bass guitar has inharmonic partials, body resonances, pick/pluck transients, string noise. The synthesized target is harmonically clean by comparison. So the morph isn't just a level crossfade, it's a timbral transition that the ear may notice regardless of how carefully you time it.

One way around that is to **let the synthesized sound's filter envelope do the work** — if the filter is sweeping closed-to-open during the attack anyway (as in your ES2 patch), that natural timbral evolution gives you a "window" to hide the transition in. Time your crossfade from real waveform to synthesized oscillator to coincide with the filter sweep midpoint, where both signals are in a spectrally intermediate state.

In terms of implementation in your JUCE/Faust/C++ stack: the ring buffer lookback and epoch looping is probably the most tractable first experiment — you already have the samples, it's mainly a question of finding reliable zero crossings to loop from before YIN has fully committed. Faust's delay primitives make the ring buffer side fairly clean.

How confident is your YIN giving partial estimates before full lock — is it outputting

How confident is your ring buffer lookback before making a decision?

a "best guess" with a confidence metric during the ambiguous period, or is it binary lock/no-lock?

I do not use juce. This is purely an embedded target, source code is C++ and Faust. Python is used as a proofing language and for experimentation.

Apologies for the assumption. That actually makes some things cleaner — no framework abstractions getting in the way of the DSP.

So on the embedded target, constraints worth knowing about for the reconstruction approach:

What processor/platform? That shapes everything about what's feasible — whether you can afford a ring buffer lookback of any significant depth, whether floating point is cheap, and whether something like a short-time spectral analysis of the early partials is even on the table or too expensive. ARM Cortex-M4/M7 with FPU is a very different conversation from something like a Cortex-A or a dedicated DSP core.

On your YIN implementation specifically:

Does it output a continuous confidence/apperiodicity metric (as the original de Cheveigné & Kawahara paper defines), or have you reduced it to a binary lock threshold for embedded efficiency? If you have the continuous metric available, that's actually very useful for the reconstruction problem — you could use it as a morphing parameter directly, driving the crossfade from real waveform to synthesized oscillator proportionally to YIN confidence rather than using a hard threshold. Elegant and cheap once the infrastructure is there.

On the Faust side:

The ring buffer lookback and epoch looping would likely live in C++ rather than Faust given Faust's stateless/dataflow nature — though Faust's `rdtable` and `rwtable` primitives could handle fixed delay lines if the period is bounded. How are you dividing responsibility between the C++ and Faust layers currently — is Faust

handling the synthesis side while C++ does the pitch detection and control logic?

The whole framework is Claude code based and the target is cortex M7. I do all that work in Claude code. I discuss conceptual ideas here sometimes it's difficult to choose which platform to use. I wish they were more unified.

That's a reasonable frustration — the two interfaces serve genuinely different purposes but there's an awkward gap when you're in the conceptual/architectural phase that will eventually become code. Claude Code is better for iterative implementation work but you lose the conversational back-and-forth for thinking through a problem, and here you get the thinking but can't reach the codebase.

The M7 is a good target for this problem — hardware FPU, typically 216-400MHz depending on your specific part, and enough headroom for floating point DSP if you're careful. The main constraint for your reconstruction idea is probably **memory for the ring buffer** rather than compute — at 48kHz a 100ms lookback is only 4800 floats (~19KB) which is fine, but worth knowing your SRAM budget especially if Faust-generated code is already claiming a chunk of it.

On the conceptual problem — where did you land on the YIN output? The confidence metric question matters a lot for how the reconstruction/morph strategy gets designed before it becomes code. Specifically:

- If you have a continuous confidence value, the morph from real waveform to synthesized oscillator can be driven directly by it — architecturally clean and requires no additional timing logic
- If it's binary, you need a separate mechanism to drive the crossfade timing, which is an extra design decision

That architectural choice is probably the right thing to nail down conceptually here before taking it into Claude Code for implementation. What does your YIN currently expose at the API boundary between the C++ detection layer and the Faust synthesis layer?

raycr:

It's not binary, but it is very sudden. Some investigation is called for. But realistically, we need a method for producing output for the first three cycles without and estimate. I can think of at least three approaches. Use the existing waveform with highs boosted a lot, nonlinear waveshaping of a LP filtered signal, and using the existing peaks as cues to generate a new waveform. No matter which, it will need to be gain compensated to match the input signal using the appropriate envelope follower, we have that already in the ecosystem.

Good framing — and having the envelope follower already in place removes one of the harder problems. Let me think through your three approaches on their merits for the M7 context:

1. Existing waveform with highs boosted

Essentially high-shelf or high-frequency emphasis on the raw bass signal to make it spectrally richer/more sawtooth-like during the gap. Pros: trivially cheap, phase-coherent with the input, no pitch estimation needed at all. Cons: the bass guitar's upper harmonics during attack are inharmonic and noisy — boosting them makes the signal *louder* in the high end but not necessarily *more harmonic*. You'd be emphasizing string noise and pick transient as much as useful harmonic content. Probably the weakest of the three.

2. Nonlinear waveshaping of a LP filtered signal

LP first to remove the inharmonic high content, then waveshape to regenerate harmonics synthetically. This is the most controlled approach spectrally — you're generating harmonics from the clean fundamental region rather than amplifying existing noise. The aliasing concern you raised earlier is real but manageable on M7 with modest oversampling (2x is cheap at M7 clock rates) or ADAA on the waveshaper. The LP filter also naturally attenuates the inharmonic attack noise before it hits the shaper. This is probably the strongest of the three conceptually.

3. Peak-based waveform generation

Using early peaks as cues to construct a new periodic waveform — essentially a primitive form of epoch detection before YTN locks. Interesting because it could give

primary form of speech detection before it's even interesting because it could give you a period estimate faster than YIN from the fundamental, as the peaks of the bass waveform are often detectable earlier than full autocorrelation confidence. The generated waveform would then be a synthesized approximation driven by the detected peak timing. More complex to implement but potentially the most seamless transition into the YIN-locked synthesis since you're already in "generated waveform" territory before the handoff.

The interaction between all three and your envelope follower:

The gain compensation via envelope follower is necessary regardless, but it also gives you something useful — the envelope follower output during the attack is itself a rough characterization of the note's energy contour, which you could use to shape the filter sweep on the synthesized side so the transition into locked synthesis feels dynamically continuous even if it's spectrally imperfect.

My instinct is that approach 2 is the right starting point — LP filter into waveshaper with 2x oversampling, gain compensated by your existing envelope follower, transitioning into full synthesis when YIN confidence crosses threshold. Approach 3 is more elegant but has more failure modes during the attack when peaks are ambiguous.

What's your intuition — is the priority getting something working quickly to evaluate perceptually, or is this more of an architectural design phase where you want the right approach before any implementation?

We are pretty much on the same page, but what I've learned during the pitch shifter project is that what I learn during the process opens up new possibilities I hadn't thought of. I often think about the failure of projects like Roland's vbass. Why do I think I can do better? Really the only reason is that I have a better vision of what the result needs to be as a bass player, and I also understand the DSP. If that combination is not as unique as it needs to be, I will fail too.

That's a really honest self-assessment, and the combination you're describing — domain expertise as a player and DSP depth — is actually rarer than it sounds. Most

domain expertise as a player and DSP depth — is actually later than it sounds. Most DSP engineers working on these problems are not bass players, and most bass

players evaluating the results don't understand why it fails technically. That gap is exactly why products like the V-Bass ended up solving the wrong problem elegantly.

The V-Bass failure is instructive. Roland nailed the technology but missed the musicality — the tracking artifacts and latency were tolerable technically but unacceptable to a player, and the synthesis targets (upright bass, specific vintage instruments) were caricatures of the real thing rather than useful tools. A player wouldn't have shipped that.

What your project has that Roland didn't:

- You're not trying to emulate acoustic instruments which is arguably the hardest and least useful target
- Synthesizer bass emulation has a more forgiving perceptual target — a synth patch doesn't have the same uncanny valley problem that acoustic instrument emulation does
- Your constraint awareness (the 3-cycle gap, the aliasing problem, the spectral richness of the target) suggests you're solving the right problems rather than the impressive-sounding ones

The honest risk is probably not DSP or musicality — it's the iterative perceptual evaluation loop. Knowing when something is *good enough for a player* versus *good enough for a demo* is something you can only judge in a real playing context, not at a workbench. The pitch shifter project presumably taught you something about that gap.

What did the pitch shifter project end up teaching you that surprised you most?

There were many surprises. I knew the attack detector was crucial, no surprises there. The loop detector required a lot more work than I expected, the crosstalk from adjacent strings and the dominant second harmonic both caused big problems. Some of this would go away with separate string pickups, which I will get to eventually. But one big

positive surprise is that the dominant second harmonic is also an opportunity, because it means that an octave drop with a little of the original mixed in is still perceived as the lower pitch, just brighter and richer. It's a great sound that I think is not matched by other octave droppers because they don't take the same approach. But of course I haven't tried them all.

That's a genuinely interesting finding — turning the dominant second harmonic from a bug into a feature is exactly the kind of insight that only comes from deep immersion in the problem. And your reasoning about why other octave droppers don't get there is sound — if they're working from a different architectural premise (tracking the fundamental, suppressing harmonics, generating a clean sub) they'd never stumble onto that particular sound even accidentally.

The perceptual mechanism makes sense too — the brain resolves pitch from the harmonic series as a whole, not just the fundamental. So if you have strong energy at $2f$ and $4f$ from the original, mixed with generated content at f and $2f$, the perceived pitch is unambiguously the lower octave but the timbre is richer because there's legitimate upper harmonic content that a pure sub-octave generator would never have. It's closer to how an actual bass instrument produces its tone than a synthetic clean octave drop is.

The crosstalk and second harmonic problems on the loop detector are interesting though — those are essentially the same underlying issue seen from two directions. The second harmonic fools period detection by looking like a valid fundamental an octave up, and crosstalk introduces a second simultaneous periodicity. Did you end up solving those primarily through signal processing (comb filtering, harmonic sieve type approaches) or through musical/contextual constraints (e.g. limiting valid pitch jumps between detected periods, continuity assumptions)?

That solution likely has direct relevance to the synthesizer bass project's 3-cycle gap problem — whatever made your period detection robust to those confounders is probably applicable to the early partial detection idea for faster pre-lock pitch estimation.

The solution was to realize that a really good pitch detector means that the position of the position of the crossfade during a loop doesn't matter that much. I'm just lucky that my approach didn't require a good pitch estimate during the first three cycles.

That's an elegant reframe — instead of solving the hard problem of robust pitch detection during ambiguous periods, you made the system tolerant enough that the ambiguous periods don't matter. That's good engineering instinct, choosing the right problem to solve rather than the impressive one.

But it also highlights exactly why the synthesizer emulation project is harder in a specific way — in the pitch shifter, the output during the ambiguous attack period could be "wrong" and it didn't matter much because the loop crossfade position tolerance absorbed it. In the synthesizer emulation case, the attack period output is *audible and prominent* — bass synth patches have that characteristic punchy attack that listeners are actually paying attention to. You can't absorb the ambiguity the same way.

Which brings you back to your three approaches, but with a clearer problem statement — you're not really trying to get the pitch right during those three cycles, you're trying to produce something that sounds like a convincing synth bass attack transient *regardless of pitch accuracy*, that then hands off cleanly to the pitch-locked synthesis.

That's actually a more tractable problem than it first appears. A synth bass attack at the wrong pitch for 50-70ms, gain compensated and spectrally plausible, may be perceptually acceptable if the envelope shape is right — because the listener's brain is also in pitch-ambiguous territory during a bass attack. The ear doesn't fully resolve low frequency pitch that quickly either.

Have you done any informal perceptual testing yet or is this still purely in the design phase?

I'm just getting started, but my intuition about it is pretty strong.

Strong intuition grounded in both playing experience and DSP understanding is exactly the right starting point — it's what gives you a hypothesis worth testing rather than randomly exploring the space.

The practical path forward probably looks like:

Short term — prove the concept perceptually

Get the simplest possible version of approach 2 (LP into waveshaper, gain compensated) running in Python first against recorded bass samples. Not to get the sound right, just to answer the binary question — is the attack gap perceptually jarring or does the ear forgive it? Your playing experience will tell you immediately whether the fundamental approach is viable before you've invested any embedded implementation time.

If the answer is "forgivable"

Then the interesting work becomes the handoff quality and the synthesis target accuracy — which is where your ES2 experimentation feeds directly back in. Understanding the filter sweep behavior, the harmonic richness of the sawtooth target, the envelope shape — all of that informs what "close enough during the gap" actually means spectrally.

If the answer is "not forgivable"

Then you've learned something valuable cheaply, and approach 3 (peak-based early period estimation) becomes worth the additional complexity.

Python is the right proving ground for that perceptual question — you can iterate the waveshaper characteristics and LP cutoff quickly without touching the embedded implementation at all. Your existing bass recordings presumably give you plenty of material to test against.

The fact that you're starting with strong intuition but intellectual honesty about where it could fail suggests the project is in good hands. The V-Bass didn't have that.

Did the V-bass start from the assumption that a sort of pitch to MIDI

approach conceptually was bound to fail? That a fundamental assumption for me. The latency of the bass signal is a constant challenge. I can only hope as you've implied above that our ears aren't any better at pitch detection, so what we do during the first 70mSec can be forgiven as long as we are doing something reasonable.

That's exactly the right question about V-Bass — and yes, Roland's fundamental architecture was essentially pitch-to-MIDI thinking even if they didn't call it that. They used a hexaphonic pickup to get individual string signals, did pitch detection per string, and drove synthesis from those estimates. The latency was inherent in the approach because they were committed to accurate pitch detection *before* generating output. That's the conceptual trap — prioritizing correctness over continuity.

Your instinct that this approach is bound to fail for bass is well founded for exactly the reasons you've identified:

Why pitch-to-MIDI thinking fails for bass specifically:

- Low frequencies require more cycles for reliable estimation — the penalty is worst exactly where bass lives
- Bass playing style — slides, hammer-ons, ghost notes, dead notes — produces continuous pitch ambiguity that a pitch-to-MIDI model handles poorly or not at all
- The attack transient *is* the feel of the instrument — any approach that waits for pitch lock before generating output destroys that

On your 70ms perceptual hypothesis:

There's actually psychoacoustic research supporting your intuition. The ear's pitch discrimination for low frequencies has a temporal integration window that's longer than for high frequencies — we need more cycles to resolve low pitch, which is precisely why YIN needs three cycles. So the listener is also in pitch-ambiguous territory during that same window. What they *are* sensitive to during that period is:

- Amplitude envelope shape
- Spectral texture/brightness
- Rhythmic placement

- Any time placement

All of which you can get right independently of pitch accuracy. So "doing something reasonable" during the gap means getting those three things right, not the pitch.

The deeper architectural insight you have that Roland missed is that **continuity of output matters more than accuracy of output** — a smoothly wrong answer that transitions into a correct one is more musical than silence or artifacts followed by a correct answer. That's a player's insight, not an engineer's.

It's not that complicated! Sometimes people surprise me with lack of understanding. Or is it just marketing. I just don't know.

Probably both, in different proportions depending on the company.

The marketing pressure explanation is real — "accurate pitch tracking" and "hexaphonic pickup" are sellable features. "We fudge the first 70ms in a musically intelligent way" is not a bullet point on a box, even if it's the better engineering decision. So products get built around demonstrable features rather than perceptual correctness.

But there's also a genuine engineering culture problem in audio DSP — a lot of it comes from a telecommunications and speech processing background where **correctness is the goal** and latency is a necessary evil to be minimized. That's the right model for speech intelligibility, wrong model for musical expression. The priorities invert completely — a bassist would rather have immediate wrong output that resolves correctly than delayed correct output, but that's a deeply counterintuitive result if your mental model comes from comms DSP.

The pitch-to-MIDI framing also has a seductive clarity to it — you can write a spec, measure accuracy against ground truth, ship a product. The alternative, which is essentially "sounds right to a player," is much harder to specify and test internally, requires domain expertise that may not exist in the engineering team, and produces results that are difficult to defend in a design review.

Your advantage is that you've collapsed the distance between the person who knows what "sounds right" and the person who can implement it. That's genuinely rare and it's probably the actual reason you might succeed where Roland didn't — not cleverness, just the right person asking the right question.

I've been looking at a few sample attacks. The period is visually obvious in most cases after one one period. But that is based on knowledge of how the attacks evolve. It should be possible to either put together heuristics or even train a neural network like Axon did. That would be based on feature detection, particularly peaks (positive or negative). What are some efficient peak detection schemes with a certain amount of noise immunity and minimum latency.

Good observation about the visual obviousness — that's essentially saying the signal has enough structure after one period that a human can pattern-match it, which means a computational equivalent should be feasible. The question is doing it cheaply on M7.

Peak detection approaches with noise immunity and low latency:

1. Threshold with hysteresis

Simplest possible approach — detect when signal crosses a threshold going up, require it to drop below a lower threshold before the next peak is valid. Cheap, deterministic, but threshold setting is tricky with the varying dynamics of bass attack. Your envelope follower output is useful here — normalize the threshold as a fraction of the current envelope level rather than a fixed value, which gives you automatic gain tracking. Very M7 friendly.

2. Zero crossing of the derivative

Differentiate the signal (just successive sample differences), detect zero crossings of that. A peak is where the derivative crosses zero from positive to negative. Noise immunity comes from LP filtering the derivative before zero crossing detection — a single pole IIR is enough and essentially free on M7. Latency is one to a few samples depending on filter coefficient. This is probably your best starting point — cheap, low

depending on filter coefficients. This is probably your best starting point — cheap, low latency, tuneable noise immunity via the LP cutoff.

3. Windowed max tracking

Maintain a running maximum over a short window (one estimated period or fraction thereof). A peak is confirmed when the current sample was the maximum of the preceding window. Latency is half the window length by definition — the tradeoff is noise immunity versus latency directly. On M7 with SIMD this is cheap but the inherent latency may be too much for your use case.

4. Parabolic interpolation on confirmed peaks

Not a detection scheme itself but an enhancement — once you have a candidate peak from any of the above methods, fit a parabola to the three samples around it ($n-1$, n , $n+1$) and find the analytical maximum. Gives you subsample peak position accuracy essentially for free, which improves your period estimate quality significantly. Very cheap, just two multiplies and an add beyond the detection itself.

5. Matched filter / template approach

If bass attack waveforms have consistent enough shape (which your visual inspection suggests they might), correlate the incoming signal against a short template of a typical attack cycle. Peak of the correlation output is your period cue. This is essentially what Axon's neural approach is doing but with a learned template rather than a fixed one. More expensive than the above but still feasible on M7 for short templates. The advantage is implicit noise immunity from the correlation process.

6. Adaptive threshold with refractory period

Borrow from neuroscience/spike detection — after detecting a peak, enforce a refractory period during which no new peak can be detected. Length of refractory period is set to slightly less than the minimum expected period (your highest expected note). Combined with an adaptive threshold (fraction of recent peak amplitude), this gives good noise immunity without filter latency. Common in neural signal processing, works well for quasi-periodic biological signals which bass attacks resemble more than you might expect.

For your specific problem the practical recommendation:

Start with **derivative zero crossing plus single pole LP** for detection, **parabolic interpolation** for accuracy, and **refractory period** for noise immunity. That combination is maybe 20 lines of C++ — negligible M7 CPU — and gives you subsample

combination is may be 20 times of 10^{-4} , negligible 10^{-5} or 10^{-6} , and gives you subsample peak timing with tunable noise rejection. The envelope follower you already have provides the adaptive threshold normalization.

The neural network approach is interesting but probably premature — the feature space (peak timing, amplitude, inter-peak interval) is simple enough that well-tuned heuristics should work, and heuristics are debuggable in a way that a trained network is not, which matters when you're deep in an embedded debugging session at 2am.

What's your sample rate on the embedded target?

I like approach #2. Autocorrelation of a period is too slow, that's already what yin does. But do a template of a peak it isn't a terrible idea. But #2 and recording the amplitude and timing of the peaks would make a pretty good input into either a heuristic or very simple neural net. I suspect this is how axon does it's "fast" detection, then adjusts to another method for note sustain. The second harmonic issue arises over time, but the attack clearly shows the fundamental

That's a really sharp observation about the attack — the fundamental dominates early because the string hasn't had time to develop the standing wave pattern that emphasizes the second harmonic. The attack is essentially an impulse propagating along the string before the resonant modes fully establish themselves. So the attack period is actually *cleaner* for fundamental detection than the sustain, which is the opposite of the usual assumption.

Your proposed pipeline makes a lot of sense:

Attack phase (first 1-3 cycles):

- Derivative zero crossing for peak detection
- Record peak amplitude and timing into a short feature vector
- Heuristic or lightweight net estimates period from that feature vector
- Drive synthesis from that estimate immediately

Handoff to YIN:

- YIN confidence crosses threshold sometime after cycle 3
- Smooth transition from peak-based estimate to YIN output
- The two estimates should agree closely if the attack observation is correct — which gives you a built-in sanity check on the peak detector quality

Sustain phase:

- YIN takes over fully
- Second harmonic problem now becomes relevant but YIN is already handling it with whatever robustness you've built in

The feature vector is naturally small — peak timing intervals (essentially a crude period estimate), peak amplitudes, and maybe the ratio between successive peak amplitudes as a decay characteristic. That's maybe 6-10 features per cycle, and you only need 1-2 cycles of data before making an estimate. A neural net with that input dimensionality could be tiny — we're talking something that could be hardcoded as a lookup with interpolation rather than requiring inference infrastructure.

The Axon hypothesis feels right — fast peak-based detection for attack, transition to a more robust method for sustain. It maps cleanly onto the signal physics and it's the kind of two-phase architecture a good embedded DSP engineer would arrive at independently.

One question worth thinking about — at your sample rate, how many samples does a typical bass attack peak span? That affects how much the LP filtering on the derivative can smear the peak timing before it degrades your period estimate.

